MULTI-CRITERIA LIBRARY RESOURCE DISTRIBUTION

Python Project Report

1.Overview

The Multi-Criteria Library Resource Distribution system is designed to distribute library resources, such as books, among students based on multiple criteria. These criteria may include student year, department, book availability, academic requirements, and borrowing limits.

The system evaluates the available resources and determines whether a student is eligible to receive a requested book. It aims to distribute library resources fairly, reduce conflicts, and improve resource utilization.

This system helps librarians manage book allocation efficiently by applying predefined rules and generating appropriate allocation decisions.

2.Example Policy Used

The following sample policies are used for library resource distribution.

1. Students must provide a valid student ID.
2. Books are allocated based on availability.
3. Each student can borrow a maximum of 3 books at a time.
4. Students with overdue books cannot borrow additional books until the overdue issue is resolved.
5. Department-related academic books may receive priority for students studying the relevant subject.
6. If multiple students request the same available book, priority is given according to the predefined allocation rules.
7. If a requested book is unavailable, the system displays "Book Not Available."
8. The system records the student ID, book ID, and allocation status.

Example:

- Student ID: S101
- Department: CSE
- Requested Book: Python Programming
- Book Availability: Available
- Books Already Borrowed: 1

- Overdue Books: 0
- Decision: Book Allocated

Note: These are illustrative policies. Actual library rules may differ.

3.Problem Statements

1. Manual distribution of library books can be time-consuming.
2. Limited book availability makes resource allocation difficult.
3. Students from different departments may request the same book.
4. Borrowing limits must be checked before allocating books.
5. Students with overdue books may require special handling.
6. Manual allocation may result in unfair distribution or human errors.
7. Libraries need an efficient system to manage resource requests based on multiple criteria.

The proposed system addresses these problems by evaluating requests using predefined allocation rules.

4.Objectives

1. To automate library resource distribution.
2. To allocate books based on multiple criteria.
3. To check book availability before allocation.
4. To enforce borrowing limits for students.
5. To identify students with overdue books.
6. To prioritize academic resources based on predefined rules.
7. To reduce errors in manual book allocation.
8. To improve fairness in resource distribution.
9. To maintain accurate allocation records.
10. To improve the efficiency of library resource management.

5.Assumptions

1. Each student has a unique student ID.
2. Each library book has a unique book ID.
3. Book availability information is updated correctly.

4. The maximum borrowing limit is predefined.
5. Student department details are available.
6. Overdue book information is maintained in the system.
7. Library allocation rules are defined in advance.
8. Each allocation request contains valid student and book details.
9. The system processes requests according to the configured policies.
10. If a book is unavailable, the system rejects the request or indicates that the student may join a waiting list.

6.Algorithm

Step 1: Start.

Step 2: Enter the student ID and requested book ID.

Step 3: Retrieve the student's department and borrowing details.

Step 4: Check whether the student ID is valid.

Step 5: Check whether the requested book is available.

Step 6: Verify whether the student has reached the borrowing limit.

Step 7: Check whether the student has overdue books.

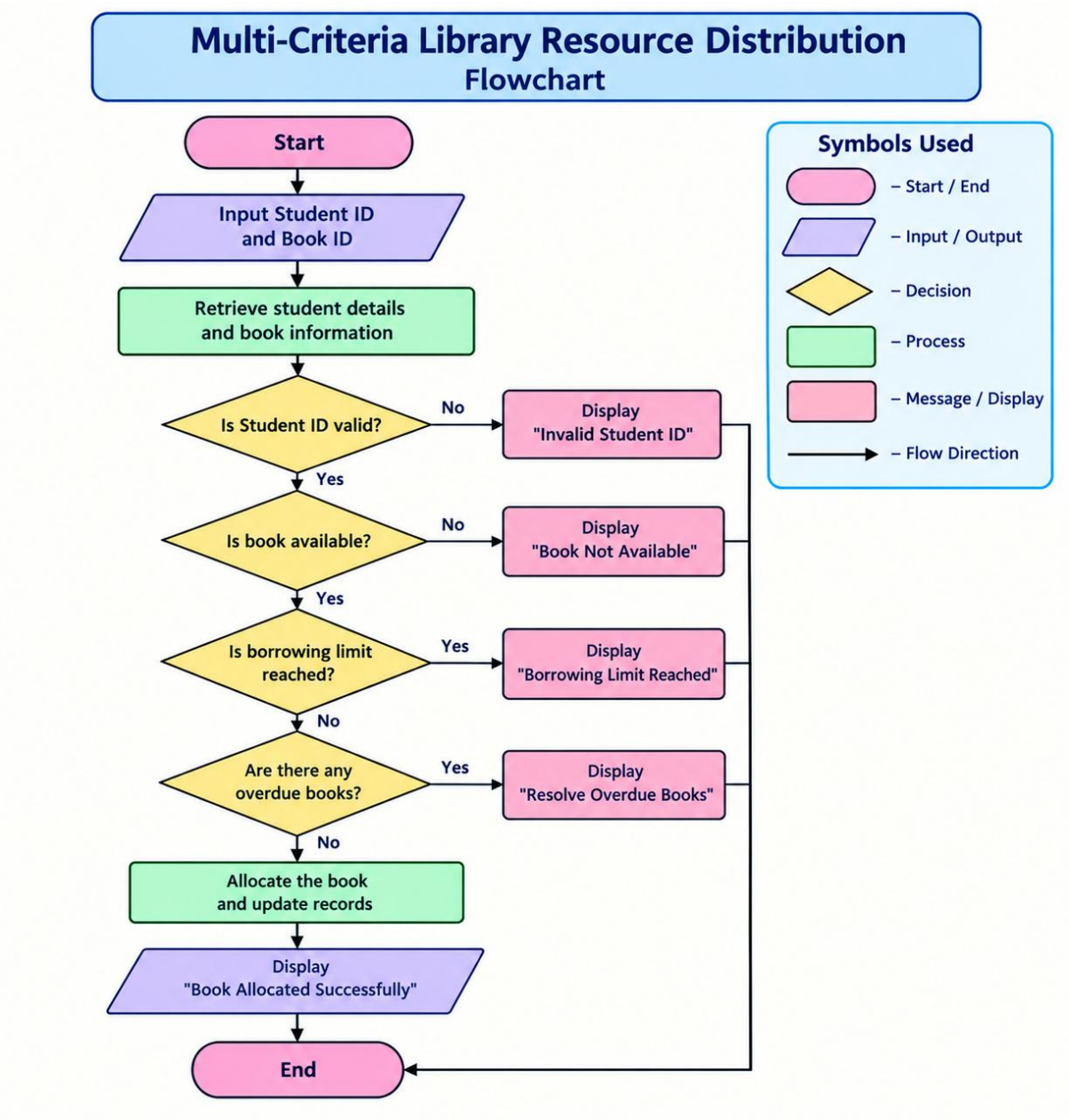
Step 8: If all eligibility conditions are satisfied, allocate the book.

Step 9: Otherwise, reject the request and display the appropriate reason.

Step 10: Update the allocation record and display the result.

Step 11: Stop.

7.Flowchart



8.Pseudocode

BEGIN

INPUT Student_ID

INPUT Book_ID

Retrieve student details

Retrieve book details

```
  IF Student_ID is invalid THEN
    DISPLAY "Invalid Student ID"

  ELSE
    IF Book is not available THEN
      DISPLAY "Book Not Available"

    ELSE
      IF Borrowed books >= Maximum limit THEN
        DISPLAY "Borrowing Limit Reached"

      ELSE
        IF Student has overdue books THEN
          DISPLAY "Resolve Overdue Books"

        ELSE
          Allocate the book to the student
          Update book availability
          Update borrowing records

          DISPLAY "Book Allocated Successfully"
        END IF
      END IF
    END IF
  END IF

END
```

9.Source Code

```
# Multi-Criteria Library Resource Distribution

# Sample student database
students = {
    "S101": {"department": "CSE", "borrowed": 1, "overdue": 0},
    "S102": {"department": "ECE", "borrowed": 3, "overdue": 0},
    "S103": {"department": "CSE", "borrowed": 1, "overdue": 0},
    "S104": {"department": "IT", "borrowed": 0, "overdue": 1}
}

# Sample library book database
books = {
    "B101": {"title": "Python Programming", "department": "CSE", "copies": 2},
    "B102": {"title": "Database Systems", "department": "CSE", "copies": 0},
    "B103": {"title": "Java Programming", "department": "IT", "copies": 2},
    "B104": {"title": "Digital Electronics", "department": "ECE", "copies": 3}
}

MAX_BOOKS = 3


def allocate_book():
    print("\n--- Library Resource Distribution System ---")

    student_id = input("Enter Student ID: ").strip().upper()
    book_id = input("Enter Book ID: ").strip().upper()
```

```
# Check student ID
if student_id not in students:
    print("Invalid Student ID")
    return

# Check book ID
if book_id not in books:
    print("Invalid Book ID")
    return

student = students[student_id]
book = books[book_id]

print("\nStudent Department:", student["department"])
print("Book Title:", book["title"])

# Check book availability
if book["copies"] <= 0:
    print("Book Not Available")
    return

# Check borrowing limit
if student["borrowed"] >= MAX_BOOKS:
    print("Borrowing Limit Reached")
    return

# Check overdue books
if student["overdue"] > 0:
```

```
        print("Cannot Allocate: Resolve Overdue Books")
        return

    # Calculate priority based on department
    if student["department"] == book["department"]:
        print("Priority: High (Department Match)")
    else:
        print("Priority: Normal")

    # Allocate book
    book["copies"] -= 1
    student["borrowed"] += 1

    print("\nBook Allocated Successfully!")
    print("Student ID:", student_id)
    print("Book Name:", book["title"])
    print("Available Copies:", book["copies"])
    print("Total Borrowed Books:", student["borrowed"])


def display_books():
    print("\n--- Available Library Books ---")
    for book_id, book in books.items():
        print(
            "Book ID:", book_id,
            "| Title:", book["title"],
            "| Department:", book["department"],
            "| Available Copies:", book["copies"]
```

```
    )


while True:
    print("\n===== MAIN MENU =====")
    print("1. Allocate Book")
    print("2. Display Books")
    print("3. Exit")

    choice = input("Enter your choice: ").strip()

    if choice == "1":
        allocate_book()
    elif choice == "2":
        display_books()
    elif choice == "3":
        print("Thank you for using the Library System!")
        break
    else:
        print("Invalid Choice. Please try again.")
```

10.Test Cases

Test Case ID	Input	Expected Output	Result
TC01	Student ID: S101, Book ID: B101	Book Allocated Successfully	Pass
TC02	Student ID: S999, Book ID: B101	Invalid Student ID	Pass

TC03	Student ID: S103, Book ID: B102	Book Not Available	Pass
TC04	Student ID: S102, Book ID: B101	Borrowing Limit Reached	Pass
TC05	Student ID: S104, Book ID: B103	Resolve Overdue Books	Pass
TC06	Student ID: S101, Book ID: B999	Invalid Book ID	Pass
TC07	Student ID: S101, Book ID: B103	Book Allocated Successfully	Pass
TC08	Student ID: S101, Book ID: B101	Department Match – High Priority	Pass
TC09	Student ID: S101, Book ID: B104	Department Mismatch – Normal Priority	Pass
TC10	Menu choice: 3	Exit the program	Pass

Test Case Description

1. **TC01:** Checks whether an eligible student can borrow an available book.
2. **TC02:** Checks whether the system rejects an invalid student ID.
3. **TC03:** Checks whether the system identifies an unavailable book.
4. **TC04:** Checks whether the borrowing limit is enforced.
5. **TC05:** Checks whether students with overdue books are prevented from borrowing.
6. **TC06:** Checks whether an invalid book ID is rejected.
7. **TC07:** Checks whether an available book can be allocated when all conditions are satisfied.
8. **TC08:** Checks whether department-related books receive high priority.
9. **TC09:** Checks whether books from another department receive normal priority.
10. **TC10:** Checks whether the user can exit the program successfully.

11.Output Screenshots

```
===== MAIN MENU =====
1. Allocate Book
2. Display Books
3. Exit
Enter your choice: 1

--- Library Resource Distribution System ---
Enter Student ID: S101
Enter Book ID: B101

Student Department: CSE
Book Title: Python Programming
Priority: High (Department Match)

Book Allocated Successfully!
Student ID: S101
Book Name: Python Programming
Available Copies: 1
Total Borrowed Books: 2
```

12.Conclusion

The **Multi-Criteria Library Resource Distribution** system provides an efficient and systematic way to manage library book allocation. It checks student details, book availability, borrowing limits, and overdue books before allocating resources. It also considers department-based priority to support fair distribution of library resources. The Python program reduces manual work, minimizes errors, and improves the efficiency of library management. Thus, this project demonstrates how programming can be used to automate library resource distribution effectively.